



EXVUL

SMART CONTRACT **AUDIT REPORT**

Vishwa Proof Token Smart Contract

JUNE 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	8
2.3 Key Findings	9
3. DETAILED DESCRIPTION OF FINDINGS	10
3.1 Zero Amount Mint/Burn Consumes Source Reference	10
3.2 Ownership Transfer Leaves Previous Owner as Active Minter	13
3.3 Arbitrary srcRef Bytes Are Not Canonically Constrained On-Chain	15
3.4 Same Owner Transfer Allowed	17
3.5 Burn Allows Zero Address Source	19
4. CONCLUSION	21
5. APPENDIX	22
5.1 Basic Coding Assessment	22
5.1.1 Apply Verification Control	22
5.1.2 Authorization Access Control	22
5.1.3 Forged Transfer Vulnerability	22
5.1.4 Transaction Rollback Attack	23
5.1.5 Transaction Block Stuffing Attack	23
5.1.6 Soft Fail Attack Assessment	23
5.1.7 Hard Fail Attack Assessment	24
5.1.8 Abnormal Memo Assessment	24
5.1.9 Abnormal Resource Consumption	24
5.1.10 Random Number Security	25
5.2 Advanced Code Scrutiny	25
5.2.1 Cryptography Security	25
5.2.2 Account Permission Control	25
5.2.3 Malicious Code Behavior	26
5.2.4 Sensitive Information Disclosure	26
5.2.5 System API	26
6. DISCLAIMER	27

7. REFERENCES	28
8. About ExVul Security	29

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Vishwa Proof Token** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Vishwa Proof Token	2026-06	Solidity

Repository

<https://github.com/vishwanetwork/lightclient-verifier>

Commit Hash

3c72f0a53737631f2cd3714fe4d3b651eb0afd2b

Contract	Address
BTCvp	0x79D154287DDC77e5C10127E68c2df1a942a330BB
Cfvc	0xa3b81AA35A2462c588639483EBE68a856Cb32b0d

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	3
LOW	2
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
MEDIUM	Zero Amount Mint/Burn Consumes Source Reference	Fixed
MEDIUM	Ownership Transfer Leaves Previous Owner as Active Minter	Fixed
MEDIUM	Arbitrary srcRef Bytes Are Not Canonically Constrained On-Chain	Acknowledged
LOW	Same Owner Transfer Allowed	Fixed
LOW	Burn Allows Zero Address Source	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Zero Amount Mint/Burn Consumes Source Reference

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

VishwaProofToken.sol

DESCRIPTION:

The vulnerable functions are `mint` and `burn`. Both functions mark a `srcRef`-derived key as processed without checking whether `amount` is greater than zero.

```
function mint(address to, uint256 amount, bytes calldata srcRef) external
    onlyMinter {
    require(to != address(0), 'mint to zero');
    bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(1)));
    require(!processed[key], 'ref already processed');
    processed[key] = true;
    balanceOf[to] += amount;
    totalSupply += amount;
}
```

In `mint`, a zero-value mint succeeds and sets `processed[key] = true`. Because the key is derived from `srcRef` and direction 1, the same mint reference can no longer be used later to mint the correct non-zero amount.

```
function burn(address from, uint256 amount, bytes calldata srcRef)
    external onlyMinter {
    bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(2)));
    require(!processed[key], 'ref already processed');
    require(balanceOf[from] >= amount, 'insufficient balance');
    processed[key] = true;
    balanceOf[from] -= amount;
    totalSupply -= amount;
}
```

The same issue exists in `burn`. Since `balanceOf[from] >= 0` is always true, a zero-value burn can consume the burn reference without reducing any balance or total supply. Later, a valid burn using the same `srcRef` will revert with `'ref already processed'`.

IMPACT:

An authorized minter can accidentally or maliciously consume valid mint or burn references with zero-amount operations. For minting, this can permanently prevent a user from receiving the tokens they are entitled to for a valid source proof. For burning, this can prevent a valid burn reference from being used later, breaking the intended accounting or redemption flow. In both cases, the idempotency mechanism records the operation as completed even though no meaningful token state change occurred.

RECOMMENDATIONS:

Add an `amount > 0` validation to both `mint` and `burn` before marking the key as processed.

```
require(amount > 0, 'zero amount');
```

The repaired implementation adds the zero-amount validation in both code paths before consuming the replay-protection key.

```
function mint(address to, uint256 amount, bytes calldata srcRef) external
    onlyMinter {
    require(to != address(0), 'mint to zero');
    require(amount > 0, 'zero amount');
    bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(1)));
    require(!processed[key], 'ref already processed');
    processed[key] = true;
    balanceOf[to] += amount;
    totalSupply += amount;
}

function burn(address from, uint256 amount, bytes calldata srcRef)
    external onlyMinter {
    require(from != address(0), 'burn from zero');
    require(amount > 0, 'zero amount');
    bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(2)));
    require(!processed[key], 'ref already processed');
    require(balanceOf[from] >= amount, 'insufficient balance');
    processed[key] = true;
```

```
    balanceOf[from] -= amount;  
    totalSupply -= amount;  
}
```

3.2 Ownership Transfer Leaves Previous Owner as Active Minter

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

VishwaProofToken.sol

DESCRIPTION:

The vulnerable function is `transferOwnership`, together with the minter role initialized in the constructor in `VishwaProofToken.sol`. The constructor makes the initial owner a minter:

```
owner = msg.sender;
isMinter[msg.sender] = true;
```

However, `transferOwnership` only updates the `owner` variable:

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), 'zero owner');
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

If the current owner transfers ownership to `newOwner`, the old owner remains in `isMinter` unless the new owner later removes it. Since `mint` and `burn` are protected by `onlyMinter`, not `onlyOwner`, the previous owner can still mint or burn tokens after ownership has supposedly been handed over. This is especially risky because the contract comments describe ownership as an MPC wallet control model, where transferring ownership may reasonably be expected to hand over operational control.

IMPACT:

After ownership transfer, the previous owner may retain permission to mint arbitrary tokens or burn user balances through `burn`. This can break the expected trust boundary of the new owner and creates a window where the old owner still has privileged token supply control. If ownership is transferred because the previous owner is compromised or being rotated out, the retained minter role can allow continued unauthorized minting or burning.

RECOMMENDATIONS:

Ownership transfer should also rotate minting authority atomically. The repaired implementation revokes the previous owner's minter role and grants the new owner a minter role in the same transaction.

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), 'zero owner');
    require(newOwner != owner, 'same owner');
    address oldOwner = owner;
    owner = newOwner;
    isMinter[oldOwner] = false;
    isMinter[newOwner] = true;
    emit MinterUpdated(oldOwner, false);
    emit MinterUpdated(newOwner, true);
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

3.3 Arbitrary srcRef Bytes Are Not Canonically Constrained On-Chain

SEVERITY:**MEDIUM****STATUS:****Acknowledged****PATH:**

VishwaProofToken.sol

DESCRIPTION:

The contract uses raw `srcRef` bytes as the replay-protection input:

```
bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(1)));  
require(!processed[key], 'ref already processed');  
processed[key] = true;
```

It does not verify whether two different `srcRef` byte values actually refer to the same underlying source event, proof, snapshot, or redemption request. For example, the same source snapshot may be encoded in multiple ways:

```
{ 'user': '0xabc', 'amount': '100', 'height': 100 }  
{ 'height': 100, 'amount': '100', 'user': '0xabc' }
```

These may represent the same proof, but their raw bytes are different. Therefore, they produce different `key` values and can both be processed.

IMPACT:

If the off-chain system generates multiple `srcRef` encodings for the same underlying proof, the same source event can be processed more than once.

RECOMMENDATIONS:

Because the contract intentionally accepts opaque `srcRef` bytes, a practical mitigation is to enforce one canonical deterministic encoding in the off-chain mint pipeline and document that invariant in the codebase. The repaired source adopts that approach by documenting that callers must provide canonical `srcRef` values and that tooling derives `srcRef` from a fixed-format snapshot seed.

```
/// @notice keccak256(srcRef, direction) => consumed. direction 1 = mint,
    2 = burn.
/// @dev srcRef is opaque on-chain (ExVul #5): the contract cannot
    canonicalize arbitrary bytes,
///      so callers MUST supply a CANONICAL, deterministic encoding for a
    given source event.
///      The mint tooling derives srcRef as sha256 of a fixed-format
    snapshot seed, which is
///      canonical per snapshot. Do not feed multiple encodings of the
    same underlying event.
mapping(bytes32 => bool) public processed;
```

This mitigation is acceptable for the current trial deployment because the trust boundary remains in the controlled mint pipeline. For a stronger long-term hardening, a future version can accept a pre-canonicalized structured digest instead of arbitrary bytes.

3.4 Same Owner Transfer Allowed

SEVERITY: LOW**STATUS:** Fixed**PATH:**

VishwaProofToken.sol

DESCRIPTION:

The vulnerable function is `transferOwnership`. The function checks that `newOwner` is not the zero address, but it does not check whether `newOwner` is already the current `owner`.

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), 'zero owner');
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

Because there is no `require(newOwner != owner)` validation, the current owner can call `transferOwnership(owner)`. This creates a successful no-op ownership transfer: the `owner` storage value remains unchanged, but the contract still emits `OwnershipTransferred(owner, owner)`.

IMPACT:

A same-owner transfer can emit a misleading `OwnershipTransferred` event without any actual state transition. Off-chain indexers, dashboards, alerting systems, or administrative scripts may incorrectly interpret the event as a completed ownership handover. This can cause operational confusion, inaccurate audit trails, or false assumptions during contract administration.

RECOMMENDATIONS:

Add a same-owner validation before emitting the event and updating storage.

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), 'zero owner');
    require(newOwner != owner, 'same owner');
}
```

```
emit OwnershipTransferred(owner, newOwner);  
owner = newOwner;  
}
```

3.5 Burn Allows Zero Address Source

SEVERITY: LOW**STATUS:** Fixed**PATH:**

VishwaProofToken.sol

DESCRIPTION:

The vulnerable function is `burn` in `VishwaProofToken.sol`. Unlike `mint`, which checks `to != address(0)`, `burn` does not validate that `from` is not the zero address.

```
function burn(address from, uint256 amount, bytes calldata srcRef)
    external onlyMinter {
        bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(2)));
        require(!processed[key], 'ref already processed');
        require(balanceOf[from] >= amount, 'insufficient balance');
        processed[key] = true;
        balanceOf[from] -= amount;
        totalSupply -= amount;
        emit Transfer(from, address(0), amount);
        emit Burned(from, amount, srcRef);
    }
```

For non-zero amounts, burning from the zero address will normally fail because `balanceOf[address(0)]` should be zero. However, combined with the missing zero-amount check, `burn(address(0), 0, srcRef)` succeeds and emits a misleading burn event from the zero address while consuming the burn reference. Even after adding `amount > 0`, explicitly rejecting `from == address(0)` keeps ERC20-style semantics consistent and prevents invalid burn requests from being treated as valid operations.

IMPACT:

Invalid burn operations from the zero address can be accepted for zero amounts and reflected in event logs. This can confuse off-chain accounting, proof processing, and reconciliation systems that consume `Burned` or `Transfer` events.

RECOMMENDATIONS:

Add a zero-address validation to `burn` before calculating or marking the processed key.

```
require(from != address(0), 'burn from zero');
```

Combined with `require(amount > 0, 'zero amount')`, the repaired implementation prevents invalid burn requests from consuming references or producing misleading events.

```
function burn(address from, uint256 amount, bytes calldata srcRef)
    external onlyMinter {
        require(from != address(0), 'burn from zero');
        require(amount > 0, 'zero amount');
        bytes32 key = keccak256(abi.encodePacked(srcRef, uint8(2)));
        require(!processed[key], 'ref already processed');
        require(balanceOf[from] >= amount, 'insufficient balance');
        processed[key] = true;
        balanceOf[from] -= amount;
        totalSupply -= amount;
        emit Transfer(from, address(0), amount);
        emit Burned(from, amount, srcRef);
    }
```

4. CONCLUSION

In this audit, we thoroughly analyzed **Vishwa Proof Token** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

The Severity field in Sections 5.1 and 5.2 indicates the potential severity of each category if such an issue were present. “Not found” means no such issue was identified in this assessment.

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found

Severity**CRITICAL**

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptographic implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-693: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About ExVul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Capital, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@ExVulSec](https://twitter.com/ExVulSec)

 **Email**
contact@exvul.com

 **Github**
github.com/ExVul-Sec